SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5

3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5
4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5

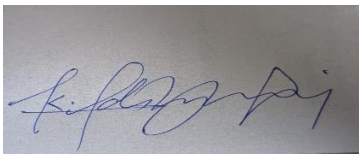
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5
9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I K ADITYA (192421015))(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS

Question 1: ATM Withdrawal Module (Black-Box Reverse Engineering)

1. Understanding of the System. Only inputs and outputs are visible: the user enters an amount and the ATM dispenses cash or shows an error. The observed messages imply these hidden rules (assumed values): a multiple of Rs.100, minimum Rs.100, maximum Rs.20,000 per transaction, Rs.50,000 per day, and not more than the balance.

2. Decomposition and Identification of Components.

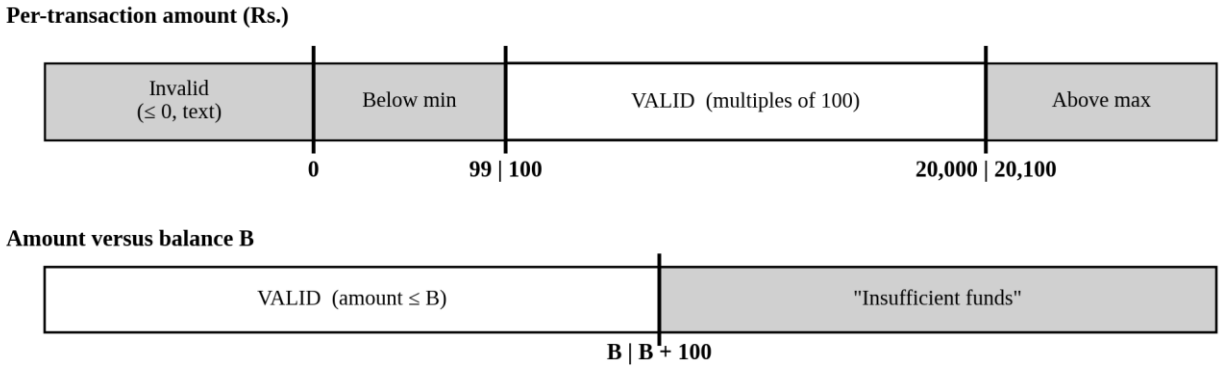
Component	Function	Equivalence classes and boundaries
Input validator	Accepts numeric multiples of 100	Invalid: text, negative, not a multiple of 100
Limit checker	Applies minimum and maximum	< 100 | 100–20,000 | > 20,000; BVA 99, 100, 20,000, 20,100
Balance validator	Compares amount with balance B	≤ B | > B; BVA B and B + 100
Daily-limit tracker	Adds up the day's withdrawals	Total ≤ 50,000 | > 50,000
Dispenser / messages	Issues notes and error text	One distinct message per failed rule

3. Reconstruction. The observed tests show equivalence partitioning (one value per class), boundary testing of the upper limit only, and error guessing for messages. Adequacy is moderate: the lower boundary, daily limit, exact-balance case, invalid input and rule combinations are untested. The improved set is:

ID	Amount	State	Expected result
TC1	100	Balance 5,000	Dispense (minimum boundary)
TC2	99	Balance 5,000	Error: below minimum
TC3	20,000	Balance 50,000	Dispense (maximum boundary)
TC4	20,100	Balance 50,000	Error: above per-transaction limit
TC5	2,550	Balance 5,000	Error: not a multiple of 100
TC6	5,000	Balance 4,900	Error: insufficient funds
TC7	–500 or "abc"	Any	Error: invalid input

ID	Amount	State	Expected result
TC8	100	Day total 50,000	Error: daily limit reached

4. Documentation. Figure 1 maps the inferred classes and boundary values to the test set above.



Question 2: Execution Traces and Code

1. Understanding of the System. The module reads n numbers, adds the even ones to sum, counts negative odd ones, and prints “High” if sum > 100, else “Low”. Only two traces exist: n = 0, and one even element, 200.

2. Decomposition and Identification of Components.

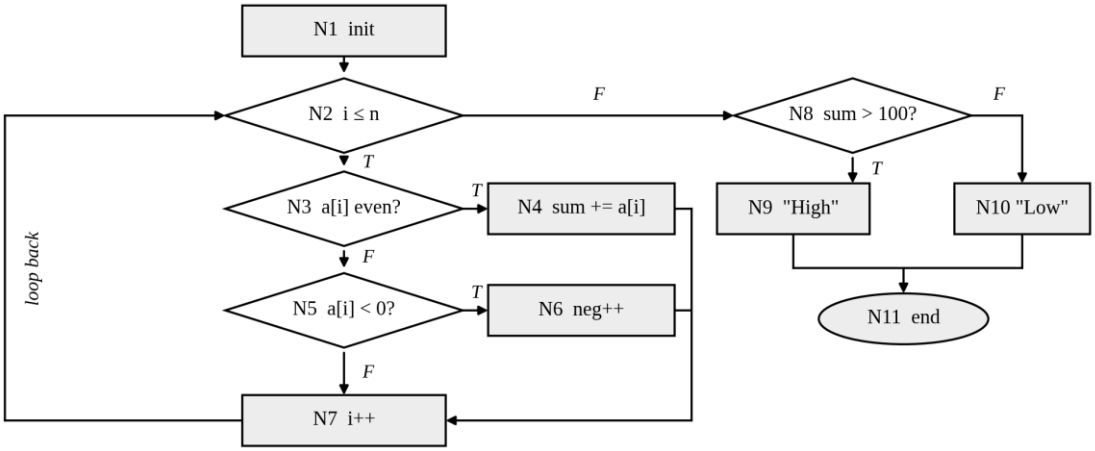
Nodes	Type	Function
N1	Initialisation	Sets sum, neg and i
N2	Loop decision (i ≤ n)	Controls iteration; back edge from N7
N3, N5	Decisions (even? negative?)	Choose the loop-body branch
N4, N6, N7	Processing	sum += a[i], neg++, i++
N8, N9, N10	Final decision and outputs	Prints “High” or “Low”

3. Reconstruction. The traces cover 9 of 11 nodes (82 % statement), 10 of 14 edges (71 % branch) and 2 of 5 basis paths. N5 and N6 are never run. The full strategy executes all five basis paths and loops zero, one and many times:

Path	Node sequence	Test data (a, n)	Expected
P1	1-2-8-10-11	n = 0	Low
P2	1-2-3-4-7-2-8-9-11	a = [200], n = 1	High
P3	1-2-3-5-6-7-2-8-10-11	a = [–3], n = 1	Low, neg = 1
P4	1-2-3-5-7-2-8-10-11	a = [3], n = 1	Low, neg = 0

Path	Node sequence	Test data (a, n)	Expected
P5	1-2-3-4-7-2-8-10-11	a = [10], n = 1	Low, sum = 10
Loop	Three iterations	a = [200, –3, 5], n = 3	High

4. Documentation. Figure 2 gives the CFG. Since V(G) = 5, paths P1–P5 give 100 % statement, branch and basis-path coverage.



E = 14, N = 11, predicates = 4 → V(G) = E − N + 2 = 5

Question 3: Web Login and Transaction System

1. Understanding of the System. Only defect logs exist. The recurring defects (assumed) are: an 8-character password rejected, lockout on the 6th instead of the 5th failed attempt, sessions not expiring on time, a zero amount accepted, and a failed payment still debiting the balance.

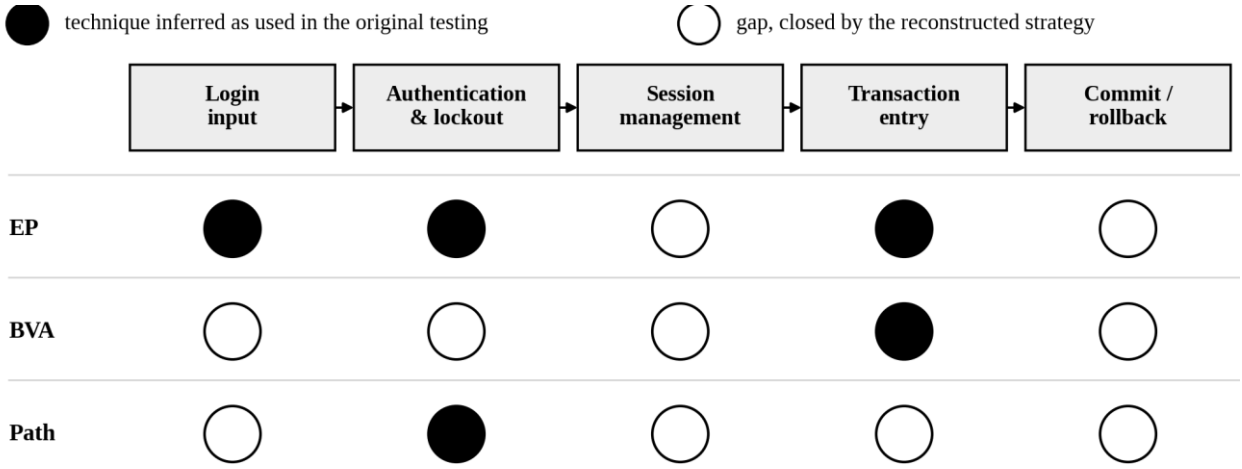
Defect pattern	Inferred technique	Meaning
Invalid and valid data classes caught	Equivalence partitioning	Class-level faults found
Off-by-one at length 8, attempt 5, amount 0	Boundary analysis missing	Faults sit at the edges
Faults after failure (rollback, expiry)	Statement testing only	Error paths not exercised

2. Decomposition and Components. The system has five testable components: login input, authentication and lockout, session management, transaction entry, and commit/rollback (Figure 3).

3. Reconstruction. The integrated strategy applies all three techniques where the gaps are:

Component	Equivalence partitioning	Boundary values	Path testing
Login input	Valid / invalid ID and password	Length 7, 8, 20, 21	Format-fail path
Authentication	Correct, wrong, locked	Attempts 4, 5, 6	Valid, invalid, lock paths
Session	Active, idle, expired	14:59 and 15:00 min	Active, expired, logout
Transaction entry	Valid, over balance	0, 0.01, B, B + 0.01	Validation-fail path
Commit / rollback	Success, failure	–	Commit, rollback, time-out

4. Documentation. EP gives broad coverage cheaply, BVA targets the off-by-one defects in the logs, and path testing reaches error branches; together they cover data and logic faults.



Question 4: Legacy Inventory System

1. Understanding of the System. The logs (assumed) show that quantity 0 or less is rejected, issuing 51 from stock 50 fails while issuing 50 succeeds, a receipt beyond 10,000 fails, and an alert appears when stock falls from 11 to 10 but not from 12 to 11.

2. Decomposition and Identification of Components.

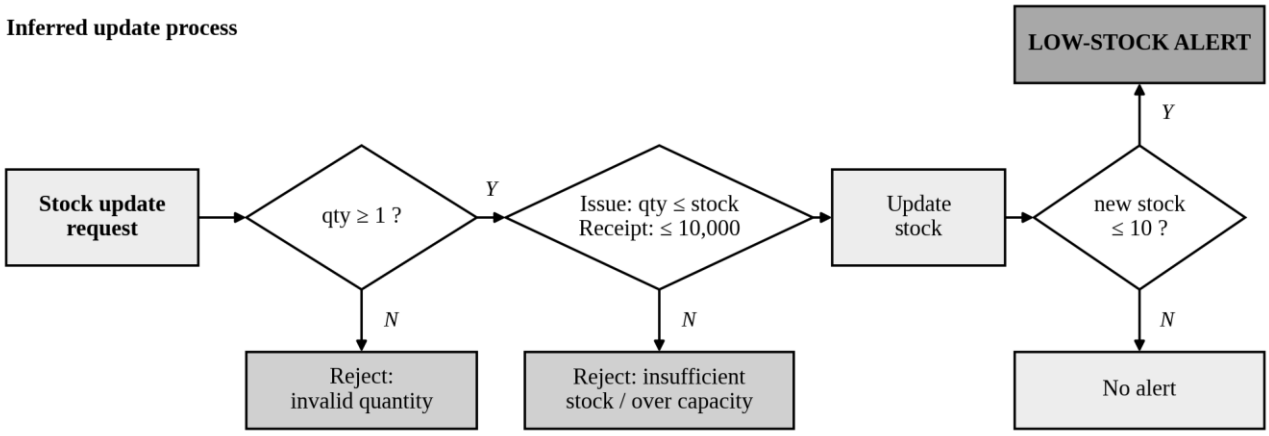
Functional unit	Function	Inferred logic
Request validator	Checks quantity	Quantity ≥ 1
Availability checker	Issue or receipt rule	Issue: qty ≤ stock; receipt: stock + qty ≤ 10,000

Functional unit	Function	Inferred logic
Stock updater	Writes the new level	New stock = stock ∓ qty
Threshold evaluator	Compares level	Alert when new stock ≤ 10
Alert dispatcher	Notifies purchasing	Triggered only by the evaluator

3. Reconstruction. The logs never show the exact edges (0, 10, 10,000) together, so each inferred rule is tested at its boundary. The rule column justifies every case:

ID	Operation	Stock	Expected	Rule tested
T1	Issue 1	50	OK, stock 49	Minimum quantity
T2	Issue 0	50	Reject	Quantity ≥ 1
T3	Issue 50	50	OK, stock 0, alert	Quantity = stock
T4	Issue 51	50	Reject	Quantity ≤ stock
T5	Issue 1	11	OK, stock 10, alert	Threshold edge
T6	Issue 1	12	OK, stock 11, no alert	Above threshold
T7	Receive 1	9,999	OK, stock 10,000	Capacity edge
T8	Receive 2	9,999	Reject	Capacity exceeded

4. Documentation. Figure 4 shows the update process reconstructed from the logs; every decision outcome is exercised by at least one test.



Question 5: Library Module .exe and CPU/Memory Profile

1. Understanding of the System. The executable is a black box, but its stress profile (conceptual, assumed) shows three bursts: S1 sustained high CPU, S2 moderate CPU with steadily rising memory, and S3 a short CPU burst. These suggest nested loops, a growing data structure and a decision cluster.

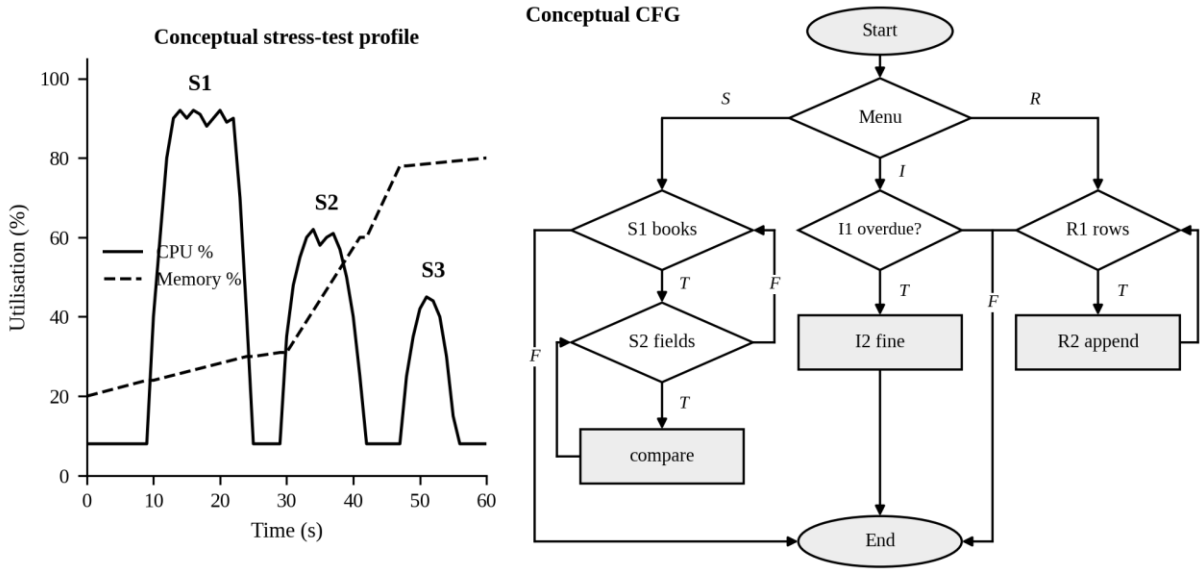
2. Decomposition and Identification of Components.

Spike	Likely logic	CFG nodes	Evidence
S1	Catalogue search: nested loop, books × fields	S1, S2, compare	CPU ≈ 90 % sustained
S2	Report generation: loop appending rows	R1, R2	CPU ≈ 60 %, memory rises
S3	Issue/return with overdue-fine decision	I1, I2	Short burst, then idle

3. Reconstruction. The path strategy targets the resource-heavy paths and records CPU, memory and response time on each:

Path	Test condition	Stability evidence gained
PS1	Search, 10,000 books, no match (S1→S2→compare)	Worst-case time of the nested loop
PS2	Search with an early match	Baseline for comparison with PS1
PS3	Issue with overdue book (I1 true, I2)	Cost of the decision cluster
PS4	Report, 50,000 rows, repeated 1,000 times	Detects memory growth or a leak
PS5	All paths mixed, one-hour soak	Overall stability under load

4. Documentation. Figure 5 pairs the profile with the CFG (V(G) = 7). The tests expose loops that cannot finish in time and memory never released; fixing them is expected to keep CPU and memory flat under stress.



Question 6: Mobile Payment Gateway

1. Understanding of the System. Each JSON response carries a status, transaction id, amount and timestamp. The triggers and constraints (assumed) define four states: Pending (initial), Success, Timeout and Reversal. The technique implicitly used is **state-transition testing**.

2. Decomposition and Identification of Components.

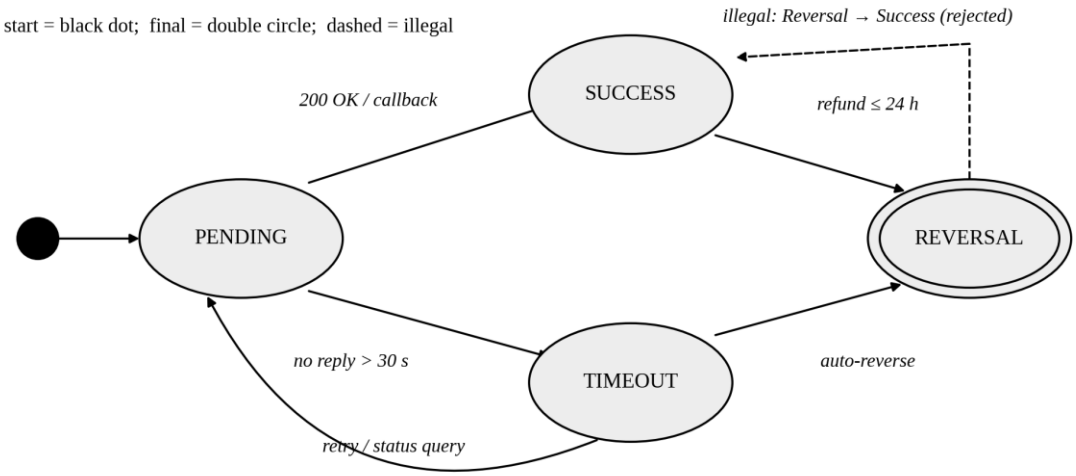
Transition	Trigger	Data constraint
Pending → Success	Callback "SUCCESS"	Same txn_id; amount matches
Pending → Timeout	No reply within 30 s	status = "TIMEOUT"
Timeout → Pending	Retry / status query	Same txn_id; retries ≤ 3
Success → Reversal	Refund request	Within 24 h; amount ≤ original
Timeout → Reversal	Auto-reversal of held funds	A debit exists

3. Reconstruction. One test per matrix cell gives 5 legal (L) and 7 illegal (X) tests, i.e. all-states and all-transitions coverage. Add T13, a one-switch sequence Pending → Timeout → Pending → Success.

From \ To	Pending	Success	Timeout	Reversal
Pending	–	L (T1)	L (T2)	X (T6)

From \ To	Pending	Success	Timeout	Reversal
Success	X (T7)	–	X (T8)	L (T3)
Timeout	L (T4)	X (T9)	–	L (T5)
Reversal	X (T10)	X (T11)	X (T12)	–

4. Documentation. Figure 6 is the state-transition diagram. Legal tests check the new status; illegal tests check that an error is returned and the state is unchanged.



Question 7: Encrypted Communication Module

1. Understanding of the System. Only input parameters and ciphertext are visible. Fuzzing with random, oversized and symbol-rich inputs (assumed results) shows: empty messages rejected, 1,024 characters accepted but 1,025 refused, keys accepted only at 16, 24 or 32 bytes, and quotes escaped or refused.

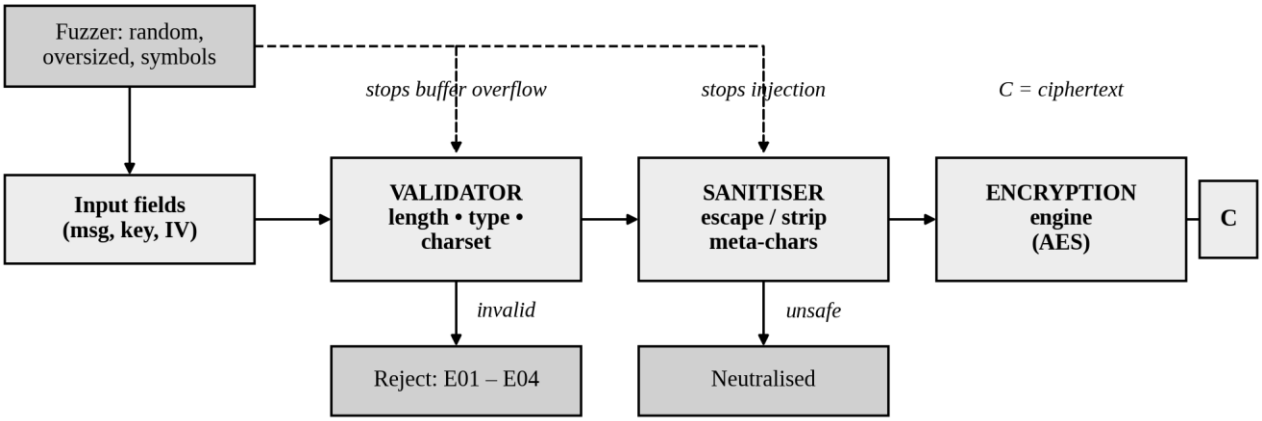
2. Decomposition and Identification of Components.

Component	Function	Inferred boundary
Data validation	Length, type and charset checks	Message 1–1,024; key 16/24/32; IV 16
Sanitiser	Neutralises meta-characters	Handles ' " ; < > and %00
Encryption engine	Encrypts validated data (AES)	Runs only after validation passes

3. Reconstruction. The improved BVA-based set tests exactly at and beyond each limit:

Field	Boundary test values	Threat prevented
Message length	0, 1, 1,024, 1,025, 65,536	Buffer overflow
Key (bytes)	15, 16, 24, 32, 33	Malformed key, memory error
IV (bytes)	15, 16, 17	Wrong block size
Content	' OR 1=1 --, <script>, %00, %n	SQL, script, format-string injection

4. Documentation and Justification. Figure 7 shows where each control sits. Boundary tests prove oversized data is rejected before it reaches the encryption buffer, preventing overflow. Content tests prove meta-characters are neutralised, preventing injection.



Question 8: Insurance Premium Calculator

1. Understanding of the System. The calculator adds a loading if age < 25 AND smoker (N2, N3) and a surcharge if age > 60 OR claims > 2 (N5, N6). The legacy suite reached 100 % statement coverage with one test (age 22, smoker, claims 3), which runs every statement but only one outcome of each condition.

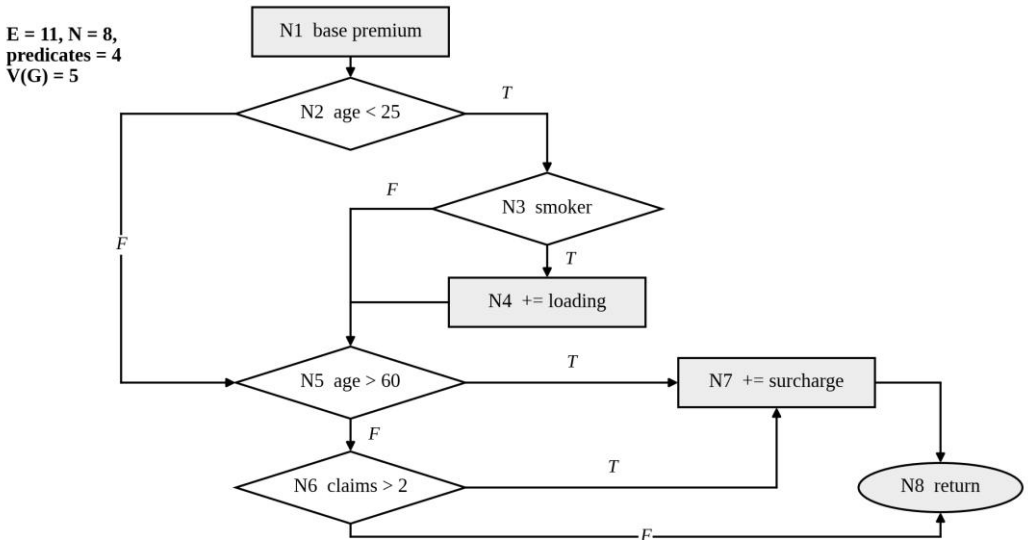
2. Decomposition and Components. The legacy approach was statement coverage only. It is inadequate because a wrong operator, such as && written for ||, still passes. The condition-level CFG (Figure 8) has 8 nodes, 11 edges and V(G) = 5.

3. Reconstruction. The two blocks give 3 × 3 = 9 routes; 2 are infeasible (age < 25 with age > 60), leaving 7 feasible paths. Running all seven gives 100 % branch and path coverage:

Test	Age	Smoker	Claims	Path (nodes)
T1	30	No	0	1-2-5-6-8
T2	30	No	3	1-2-5-6-7-8

Test	Age	Smoker	Claims	Path (nodes)
T3	65	No	0	1-2-5-7-8
T4	22	No	0	1-2-3-5-6-8
T5	22	No	3	1-2-3-5-6-7-8
T6	22	Yes	0	1-2-3-4-5-6-8
T7	22	Yes	3	1-2-3-4-5-6-7-8

4. Documentation. Every condition now takes both outcomes in every combination, so a wrong operator, limit or missing loading changes the premium in at least one test, which lowers the risk of calculation errors.



Question 9: Cloud File Synchronisation Tool

1. Understanding of the System. The tool uploads files in chunks and verifies them. The defect reports (assumed) show lost files mainly for files over 2 GB on slow or unstable networks, for 0-byte files, and for temporary or long-named files treated as conflicts.

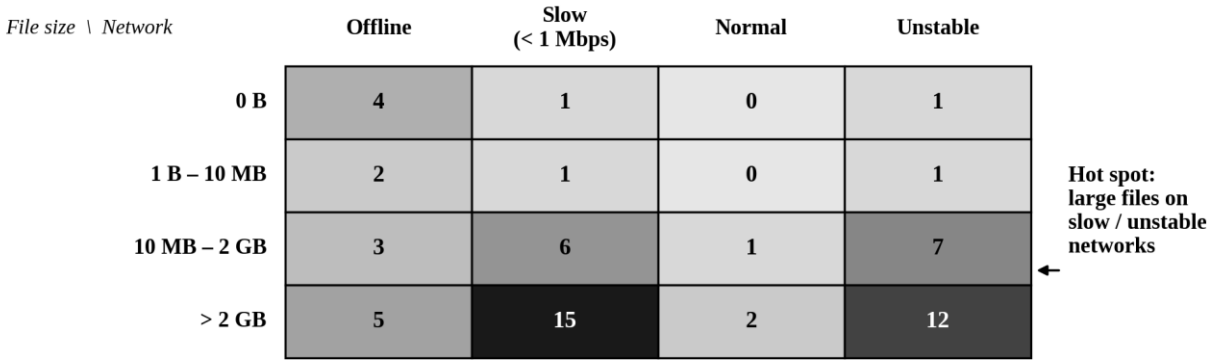
2. Decomposition and Identification of Components. Inferred partitions (ECP):

Factor	Equivalence classes	Boundary values
File size	0 B | 1 B–10 MB | 10 MB–2 GB | > 2 GB	0, 1 B, 10 MB ± 1, 2 GB ± 1
File type	Normal | temporary/hidden | long or special name	Name of 255 and 256 characters
Network	Offline | < 1 Mbps | normal | unstable	0, 0.9, 1.0 Mbps; drop mid-chunk

3. Reconstruction. The revised strategy applies BVA to size and name length, then repeats each boundary under network stress:

ID	Boundary value	Stress condition	Expected result
S1	0 B	Normal	Synced, not skipped
S2	2 GB – 1, 2 GB, 2 GB + 1	0.5 Mbps throttle	All synced; checksum matches
S3	2 GB + 1	Disconnect at 50 %	Resumes; nothing lost
S4	10 MB + 1	1,000 files at once	None lost; queue ordered
S5	Name of 256 characters	Normal	Clear error; local copy kept
S6	Temporary file	Offline, then online	Synced or flagged, not deleted
S7	5 GB	24 h soak, unstable	Zero lost files

4. Documentation. Figure 9 shows the defect hot spot. Checksums after each chunk, resumable uploads, and deleting the local copy only after server acknowledgement prevent future data loss; the stress tests prove these safeguards at every boundary.



Question 10: E-commerce Search and Filter Module

1. Understanding of the System. The SQL queries (assumed) show that each selected filter adds one clause: brand uses IN (b1 OR b2), price uses BETWEEN, rating uses ≥ 4, and the stock filter uses NOT (stock = 0). Clauses are joined by AND; unselected filters add nothing.

2. Decomposition and Identification of Components. Four filters (F1–F4) feed one query builder, and the action is the set of WHERE clauses produced (Figure 10).

3. Reconstruction. The decision table below covers every Y/N combination of the four conditions (16 rules, one test each); Rule 16 returns all rows and Rule 1 returns the smallest set.

Rule	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
C1 Brand	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N	N	N	N	N	N
C2 Price	Y	Y	Y	Y	N	N	N	N	Y	Y	Y	Y	N	N	N	N
C3 Rating	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
C4 In stock	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
Clauses	4	3	3	2	3	2	2	1	3	2	2	1	2	1	1	0

The extension tests below cover the logic inside each condition:

Extension	Test values	Expected result
Brand OR	b1 only; b1 and b2; neither matches	Union of brands; empty set
NOT stock	stock = 0 and stock = 1	0 excluded; 1 included
Invalid range	Price a > b	Empty set or validation error

4. Evaluation of Completeness. The model is complete for the four binary filters (16 of 16 rules, 100 %). It does not yet model NULL ratings, sorting or paging, so a second table is recommended for those inputs.

